

CUSTOMER SUPPORT TICKET ANALYZER USING PYTHON

Project Overview

Customer Support Ticket Analyzer is a Python-based project designed to store, clean, analyze, and extract insights from customer support tickets. The project works with customer names, issue descriptions, ticket priorities, and ticket numbers.

The project demonstrates Python fundamentals including lists, dictionaries, loops, conditionals, functions, string operations, sets, and basic data analysis.

Problem Statement

Customer support teams handle numerous service tickets daily. Analyzing support tickets helps identify common issues, customer feedback patterns, support quality, and areas for improvement.

The objective of this project is to build a Python-based Ticket Analysis System that stores, cleans, analyzes, and extracts insights from customer support tickets.

Objectives

- Store customer support ticket information using a dictionary of lists.
- Allow users to add new support tickets.
- Automatically assign ticket numbers to new tickets.
- Validate ticket priority as High, Medium, or Low.
- Clean issue descriptions using Python string operations.
- Perform keyword-based analysis of ticket descriptions.
- Analyze the distribution of ticket priorities.
- Identify the ticket with the longest issue description.
- Extract and sort unique words used in issue descriptions.

Technologies and Concepts Used

Technology / Concept	Usage
Python	Core programming language
Lists	Store ticket numbers, customer names, descriptions, and priorities
Dictionary	Organize ticket data using key-value pairs
For Loop	Iterate through tickets and descriptions
While Loop	Validate ticket priority
Functions	Reusable text-cleaning and keyword-counting logic
String Operations	Lowercasing, replacing characters, splitting, and joining text
Set	Store unique words
Conditionals	Apply priority validation and analysis logic

Project Workflow

1. Load the initial 10 customer support tickets.
2. Display the initial ticket data.
3. Ask the user how many new tickets should be added.
4. Collect customer name, issue description, and priority for each new ticket.
5. Validate the priority and append the new ticket information.
6. Clean all issue descriptions.
7. Count tickets containing selected keywords.
8. Display the final cleaned ticket data.
9. Analyze High, Medium, and Low priority tickets.
10. Find the ticket with the longest issue description.
11. Extract and sort unique words.

Initial Dataset

The project begins with 10 predefined tickets. Each ticket contains a ticket number, customer name, issue description, and priority.

Ticket No.	Customer	Issue Description	Priority
1	Ravi	Internet not working!!!	High
2	Meera	slow response, very poor service	Low
3	Sam	GREAT support! issue resolved.	High
4	Anu	okay... need help	Medium
5	Rakesh	not BAD but slow	Low
6	Divya	Excellent guidance, Very Helpful!	High
7	Arjun	good support and good behaviour!	Medium
8	Kiran	Poor handling of technical issue	High
9	Leela	Satisfied. Could be better.	Low
10	Nisha	Good service... quick response.	Medium

Data Addition

The user can add any number of new tickets. New ticket numbers automatically start from 11 and increment by one for each additional ticket.

Priority validation accepts only High, Medium, or Low. If an invalid value is entered, the program asks the user to enter a valid priority.

Text Cleaning

The issue descriptions are cleaned using the following operations:

- Convert all text to lowercase.
- Remove punctuation such as periods, commas, exclamation marks, question marks, and hyphens.
- Replace shorthand such as "ok" with "okay".
- Convert multiple spaces into a single space.
- Remove leading and trailing spaces.

Keyword Analysis

A user-defined function named `count_tickets_with_word(word)` performs a case-insensitive keyword search. It checks the cleaned issue descriptions and returns the number of tickets containing the specified word.

The assignment requires analysis of these keywords:

- poor
- good
- slow
- excellent

Priority Analysis

The final dataset is analyzed to determine the number of High-priority, Medium-priority, and Low-priority tickets.

Longest Issue Description

The program counts the words in every cleaned issue description and identifies the ticket with the highest word count. The final output displays the ticket number, customer name, cleaned issue description, and word count.

Unique Word Analysis

All words from the cleaned issue descriptions are collected into a set. Because sets contain unique values, duplicate words are automatically removed. The program then displays the number of unique words and the sorted list of unique words.

Outputs

- Initial ticket data
- Updated ticket data after adding new tickets
- Cleaned issue descriptions
- Keyword counts for poor, good, slow, and excellent
- Final cleaned ticket data
- High, Medium, and Low priority ticket counts
- Longest issue description details

- Number and sorted list of unique words

Learning Outcomes

- Working with Python lists and dictionaries.
- Adding and modifying data using `append()`.
- Using `for` and `while` loops.
- Using conditional statements for validation and analysis.
- Creating and calling user-defined functions.
- Applying string manipulation techniques.
- Using sets to identify unique values.
- Performing basic text-based data analysis.
- Presenting analytical results clearly.

Conclusion

The Customer Support Ticket Analyzer demonstrates how Python can be used to organize customer support data, clean unstructured text, perform keyword-based analysis, and generate basic analytical insights. The project provides practical experience with Python data structures, loops, functions, string processing, and summary analysis.